

고급C프로그래밍

- 코딩테스트 과제보고서 -

수 0,1,2 새빛관 303호

컴퓨터정보공학부

2024402055

박현지

No 1. 관리자 모드, 서비스 모드 진입 기능

1. 함수 설계

1) void main()

로그인 처리 위한 check_ID() 함수 호출, 동작 모드 출력

2) int check_ID()

사용자에게 ID를 입력 받음

등록된 아이디가 아닐 경우 다시 입력받고, 오류 메시지 출력

ID가 "admin"일 경우 0 반환, "user"일 경우 1 반환

2. 데이터 설계

1) char mode

사용자에게 입력받은 ID. "admin", "user"와 대조할 때 사용

2) int mode

check_ID()로부터 반환받은 변수, 모드 진입 시 사용

3. 프로그램 동작 흐름

- 1) check_ID() 함수에서 while문으로 반복
- 2) char mode에 사용자로부터 ID 입력
- 3) if문 통해 "admin", "user"와 비교
- 4) 둘 다 아닐 경우 오류 메시지 출력 후 다시 아이디 입력
- 5) "admin" 혹은 "user"와 일치할 경우 각각 0,1 반환
- 6) main()에서 진입한 모드 출력
- 7) 실행 결과

```
Mode Select (admin or user) : admin
Admin mode
```

```
Mode Select (admin or user) : user
User mode
```

4. 제약 사항

- 1) char mode 최대 길이가 10. 더 긴 ID입력에 문제가 있음

No2. 서비스 모드#1. 주문 메뉴 출력 기능

1. 함수 설계

1) void main()

로그인 처리 위한 check_ID() 함수 호출, 동작 모드 출력

User Mode 접속시 아래 3, 4번 함수 호출하여 메뉴 목록 초기화

service_mode() 함수 호출하여 주문 진행, 동적할당 해제

2) int check_ID()

사용자에게 ID를 입력 받음

등록된 아이디가 아닐 경우 다시 입력받고, 오류 메시지 출력

ID가 "admin"일 경우 0 반환, "user"일 경우 1 반환

3) void reset_drink_menu()

drink1.txt 파일에서 음료 메뉴 정보 읽은 뒤, 각 메뉴의 ID와 이름을 구조체에 목록으로 저장

4) void reset_snack_menu ()

snack1.txt 파일에서 간식 메뉴 정보 읽은 뒤, 각 메뉴의 ID와 이름을 구조체에 목록으로 저장

5) void service_mode()

저장된 음료 메뉴와 간식 메뉴를 출력한 뒤, ID를 입력받아 주문 받음. 주문을 끝내면 영수증(주문한 메뉴명, 메뉴의 가격, 총 금액)을 화면에 출력

2. 데이터 설계

1) char mode

사용자에게 입력받은 ID. "admin", "user"와 대조할 때 사용

2) int mode

check_ID()로부터 반환받은 변수, 모드 진입 시 사용

3) char tmp

reset_drink_menu()와 reset_snack_menu() 함수에서 파일을 읽어올 버퍼로 사용함. fgets를 통해 한 줄 저장

4) struct menu (==Menu)

메뉴의 ID(int id)와 이름(char* name)을 저장하는 구조체

5) Menu* drink_list, snack_list, receipt_list

4번 구조체 형식의 포인터형(배열)으로, 각각 음료, 간식, 영수증의 정보를 저장. 필요한 구조체 개수에 따라 동적할당으로 저장(realloc으로 확장)

각 메뉴의 이름(name)은 동적할당으로 저장함. int drink_cnt, snack_cnt, receipt_cnt는 각 구조체 배열의 개수 및 위치 나타냄

6) int opt, menu_opt

각각 음료, 간식, 구매 중 선택하는 변수와, 메뉴의 ID(id)를 선택하는 변수

3. 프로그램 동작 흐름

- 1) check_ID() 함수에서 while문으로 반복
- 2) char mode에 사용자로부터 ID 입력
- 3) if문 통해 "admin", "user"와 비교
- 4) 둘 다 아닐 경우 오류 메시지 출력 후 다시 아이디 입력
- 5) "admin" 혹은 "user"와 일치할 경우 각각 0,1 반환
- 6) main()에서 진입한 모드 출력
- 7) "user"입력을 통해 서비스 모드 접속 시 reset-함수 통해 메뉴 목록 초기화
- 8) 메뉴 목록 화면에 출력 후 음료(1), 간식(2), 구매(3) 선택 옵션 출력 및 입력
- 9) 1 혹은 2 선택 시 메뉴의 ID(id) 통해 영수증에 정보 순서대로 저장
- 10) 메뉴에 없는 번호 선택 시 오류 메시지 출력
- 11) 3 선택까지 반복, 그때까지 선택한 메뉴(영수증) 화면에 출력함
- 12) 실행 결과

```
Mode Select (admin or user) : user
User mode
===== drink Menu =====
1 : Americano
2 : Cafe Latte
3 : Cappuccino
4 : Green Tea
5 : Black Tea
6 : Lemonade
7 : Orange Juice
8 : Cola
9 : Sprit

===== snack Menu =====
1 : Cheesecake
2 : Chocolate Cake
3 : Croffle
4 : Muffin
5 : Cookie
6 : Sandwich
7 : Bagel
8 : Pretzel
9 : Browni
```

```
Type 1 for drink, 2 for snack, 3 for buy : 1
Type drink id to buy : 2

Type 1 for drink, 2 for snack, 3 for buy : 2
Type snack id to buy : 3

Type 1 for drink, 2 for snack, 3 for buy : 2
Type snack id to buy : 3

Type 1 for drink, 2 for snack, 3 for buy : 4
Error : Wrong input for type select!
Type 1 for drink, 2 for snack, 3 for buy : 1
Type drink id to buy : 14
Error : Not enough ID for drink
Type drink id to buy : 2

Type 1 for drink, 2 for snack, 3 for buy : 3
===== Receipt =====
Cafe Latte
Croffle
Croffle
Cafe Latte
```

4. 제약 사항

- 1) char mode 최대 길이가 10. 더 긴 ID입력에 문제가 있음
- 2) 메뉴에 재고 기능이 없음. 이론상 무한대로 추가 가능. 재고 고려 기능이 필요함

No3. 서비스 모드#2. 재고 관리 및 영수증 발급 기능 개발

1. 함수 설계

1) void main()

로그인 처리 위한 check_ID() 함수 호출, 동작 모드 출력
User Mode 접속시 아래 3, 4번 함수 호출하여 메뉴 목록 초기화
service_mode() 함수 호출하여 주문 진행, 동적할당 해제

2) int check_ID()

사용자에게 ID를 입력 받음
등록된 아이디가 아닐 경우 다시 입력받고, 오류 메시지 출력
ID가 "admin"일 경우 0 반환, "user"일 경우 1 반환

3) void reset_drink_menu()

drink2.txt 파일에서 음료 메뉴 정보 읽은 뒤, 각 메뉴의 ID와 이름, 가격과 재고를 구조체에 목록으로 저장. 필요한 개수만큼 동적할당으로 구조체 확장.

fgets를 통해 한 줄 씩 읽은 뒤, tap('Wt')과 enter('Wn')를 기준으로 이름, 가격, 재고 정보를 분리 후 구조체에 저장. 이때 저장되는 메뉴 이름은 길이에 맞춰 동적할당으로 저장.

4) void reset_snack_menu ()

snack2.txt 파일에서 간식 메뉴 정보 읽은 뒤, 각 메뉴의 ID와 이름, 가격과 재고를 구조체에 목록으로 저장. 이하 내용은 3(reset_drink_menu())함수와 동일함.

5) void service_mode()

저장된 음료 메뉴와 간식 메뉴들의 ID와 이름, 가격과 재고를 목록 양식으로 출력하고 주문을 받으며, 주문받은 메뉴의 금액만큼 총 금액 (total_cost)에 더함. 주문받은 메뉴의 재고를 차감함. 주문 종료 시 영수증에 저장함. print_receipt() 함수를 호출한다.

6) void print_receipt()

service_mode()에서 주문한 메뉴명과 금액을 출력하고, 총 금액을 계산하여 화면에 출력한다. 추가로 해당 정보를 receipt.txt에 저장한다.

2. 데이터 설계

1) **char mode**

사용자에게 입력받은 ID. "admin", "user"와 대조할 때 사용

2) **int mode**

check_ID()로부터 반환받은 변수, 모드 진입 시 사용

3) **char tmp**

reset_drink_menu()와 reset_snack_menu() 함수에서 파일을 읽어올 버퍼로 사용함.
fgets를 통해 한 줄 저장

4) **char tmp_name, tmp_cost, tmp_inven**

파일에서 읽어온 tmp 버퍼에서 각각 이름, 가격, 재고로 분리하여 임시 저장함. 이후, 구조체의 각 요소로 저장함. int name_cnt, cost_cnt, inven_cnt는 각 문자열의 길이, 위치를 알기 위한 임시 변수

5) **struct menu (==Menu)**

메뉴의 ID(int id)와 이름(char* name), 가격(int cost), 재고(int inventory)를 저장하는 구조체

6) **Menu* drink_list, snack_list, receipt_list**

4번 구조체 형식의 포인터형(배열)으로, 각각 음료, 간식, 영수증의 정보를 저장. 필요한 구조체 개수에 따라 동적할당으로 저장(realloc으로 확장)

각 메뉴의 이름(name)은 동적할당으로 저장함. int drink_cnt, snack_cnt, receipt_cnt는 각 구조체 배열의 개수 및 위치 나타냄

7) **int opt, menu_opt**

각각 음료, 간식, 구매 중 선택하는 변수와, 메뉴의 ID(id)를 선택하는 변수

8) **int total_cost**

구매한 메뉴의 총 금액을 저장하는 변수

3. 프로그램 동작 흐름

1) check_ID() 함수에서 while문으로 반복

2) char mode에 사용자로부터 ID 입력

3) if문 통해 "admin", "user"와 비교

4) 둘 다 아닐 경우 오류 메시지 출력 후 다시 아이디 입력

- 5) "admin" 혹은 "user"와 일치할 경우 각각 0,1 반환
- 6) main()에서 진입한 모드 출력
- 7) "user"입력을 통해 서비스 모드 접속 시 reset_drink_menu(), reset_snack_menu() 함수를 통해 메뉴 초기화
- 8) 메뉴 목록 화면에 출력 후 음료(1), 간식(2), 구매(3) 선택 옵션 출력 및 입력
- 9) 1 혹은 2 선택 시 메뉴의 ID(id) 통해 영수증에 정보 순서대로 저장
- 10) 메뉴에 없는 번호 선택 시 오류 메시지 출력
- 11) 3번 선택할 때까지 반복
- 12) 재고가 없을 경우 별도 메시지 출력.
- 13) 3 선택 시 지금까지 선택한 메뉴 해당 메뉴의 금액, 그리고 총 금액을 계산하여 화면에 출력.
- 14) 같은 내용의 영수증 receipt.txt에 기록, 저장
- 15) 실행 결과

```
Mode Select (admin or user) : user
User mode
===== drink Menu =====
1 : Americano 3000 15
2 : Cafe Latte 4000 12
3 : Cappuccino 4500 10
4 : Green Tea 2500 8
5 : Black Tea 2500 5
6 : Lemonade 4000 2
7 : Orange Juice 3500 9
8 : Cola 2000 20
9 : Sprite 2000 18
===== snack Menu =====
1 : Cheesecake 5500 6
2 : Chocolate Cake 5500 8
3 : Croffle 4000 10
4 : Muffin 3000 15
5 : Cookie 2500 20
6 : Sandwich 4500 7
7 : Bagel 3500 12
8 : Pretzel 3000 9
9 : Brownie 4000 5
```

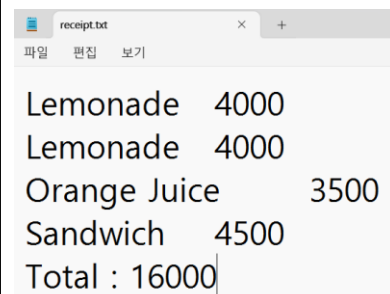
```
Type 1 for drink, 2 for snack, 3 for buy : 1
Type drink id to buy : 6

Type 1 for drink, 2 for snack, 3 for buy : 1
Type drink id to buy : 6

Type 1 for drink, 2 for snack, 3 for buy : 1
Type drink id to buy : 6
Out of stock!
Type drink id to buy : 7

Type 1 for drink, 2 for snack, 3 for buy : 2
Type snack id to buy : 6

Type 1 for drink, 2 for snack, 3 for buy : 3
===== Receipt =====
Lemonade 4000
Lemonade 4000
Orange Juice 3500
Sandwich 4500
Total : 16000
=====
```



```
receipt.txt
파일 편집 보기
Lemonade 4000
Lemonade 4000
Orange Juice 3500
Sandwich 4500
Total : 16000
```

4. 제약 사항

- 1) char mode 최대 길이가 10. 더 긴 ID입력에 문제가 있음
- 2) 프로그램 여러 번 실행 시 재고에 변화가 있었으면 함. 프로그램 종료 전, 구매 후 차감된 재고를 저장하는 코드 추가 작성 필요함
- 3) receipt.txt에 저장되는 양식 보다 가독성 있게 변경할 필요 있음

No4. 관리자 모드. 재고 주문 기능 개발

1. 함수 설계

1) void main()

로그인 처리 위한 check_ID() 함수 호출, 동작 모드 출력

asset.txt 파일에서 보유금 불러오기

아래 3, 4번 함수 호출하여 메뉴 목록 초기화

User Mode 접속시 sevice_mode() 함수 호출하여 주문 진행

Admin Mode 접속시 admin_mode() 함수 호출하여 재고 확인

모든 기능을 끝낸 후 변경된 정보(재고, 보유금)를 각각 drink2.txt, snack2.txt, asset.txt 파일에 저장함

동적할당 해제

2) int check_ID()

사용자에게 ID를 입력 받음

등록된 아이디가 아닐 경우 다시 입력받고, 오류 메시지 출력

ID가 "admin"일 경우 0 반환, "user"일 경우 1 반환

3) void reset_drink_menu()

drink2.txt 파일에서 음료 메뉴 정보 읽은 뒤, 각 메뉴의 ID와 이름, 가격과 재고를 구조체에 목록으로 저장. 필요한 개수만큼 동적할당으로 구조체 확장.

fgets를 통해 한 줄 씩 읽은 뒤, tap('Wt')과 enter('Wn')를 기준으로 이름, 가격, 재고 정보를 분리 후 구조체에 저장. 이때 저장되는 메뉴 이름은 길이에 맞춰 동적할당으로 저장.

4) void reset_snack_menu ()

snack2.txt 파일에서 간식 메뉴 정보 읽은 뒤, 각 메뉴의 ID와 이름, 가격과 재고를 구조체에 목록으로 저장. 이하 내용은 3(reset_drink_menu())함수와 동일함.

5) void service_mode()

저장된 음료 메뉴와 간식 메뉴들의 ID와 이름, 가격과 재고를 목록 양식으로 출력하고 주문을 받으며, 주문받은 메뉴의 금액만큼 총 금액 (total_cost)에 더함. 주문받은 메뉴의 재고를 차감함. 주문 종료 시 영수증에 저장함. print_receipt() 함수를

호출함.

6) void print_receipt()

service_mode()에서 주문한 메뉴명과 금액을 출력하고, 총 금액을 계산하여 화면에 출력한다. 추가로 해당 정보를 receipt.txt에 저장한다. 보유금액(asset)에 총금액을 더함.

7) void admin_mode()

음료와 간식 목록에서 재고가 0인 것(품절메뉴)을 order_list 구조체에 동적할당을 통해 저장함. 품절된 메뉴를 출력함. 없을 경우 "None" 출력. store을 통해 구조체 동적할당. 각 외부업체(gmarket.txt, coupang.txt, 11st.txt) 파일에서 정보를 초기화, 저장, 출력. 저장 방법은 위 3, 4번 reset-함수와 동일함. 외부업체 상품 목록에서 품절된 메뉴와 동일하고, 재고가 5개 이상인 것을 찾은 뒤, 가격을 비교하여 저렴한 것으로 주문 목록에 저장. 최솟값을 order_list의 cost에 저장 후, 비교하는 방식 사용. order_print() 함수 호출

8) void order_print()

주문 목록에 있는 금액으로 5개 구매함. 해당 금액만큼 보유금액(asset)에서 차감, 각 음료, 간식 목록의 재고는 늘어남. 주문 목록과 차감된 보유금액 화면에 출력.

2. 데이터 설계

1) char mode

사용자에게 입력받은 ID. "admin", "user"와 대조할 때 사용

2) int mode

check_ID()로부터 반환받은 변수, 모드 진입 시 사용

3) char tmp

reset_drink_menu()와 reset_snack_menu() 함수에서 파일을 읽어올 버퍼로 사용함. fgets를 통해 한 줄 저장

4) char tmp_name, tmp_cost, tmp_inven

파일에서 읽어온 tmp 버퍼에서 각각 이름, 가격, 재고로 분리하여 임시 저장함. 이

후, 구조체의 각 요소로 저장함. int name_cnt, cost_cnt, inven_cnt는 각 문자열의 길이, 위치를 알기 위한 임시 변수

5) struct menu (==Menu)

메뉴의 ID(int id)와 이름(char* name), 가격(int cost), 재고(int inventory)를 저장하는 구조체

6) Menu* drink_list, snack_list, receipt_list, order_list, store

4번 구조체 형식의 포인터형(배열)으로, 각각 음료, 간식, 영수증, 주문 목록, 외부업체 목록의 정보를 저장. 필요한 구조체 개수에 따라 동적할당으로 저장(realloc으로 확장). 각 메뉴의 이름(name)은 동적할당으로 저장함. int drink_cnt, snack_cnt, receipt_cnt, order_cnt는 각 구조체 배열의 개수 및 위치 나타냄.

7) int opt, menu_opt

각각 음료, 간식, 구매 중 선택하는 변수와, 메뉴의 ID(id)를 선택하는 변수

8) int total_cost

구매한 메뉴의 총 금액을 저장하는 변수

9) int asset

보유 금액을 저장하는 함수

3. 프로그램 동작 흐름

- 1) asset.txt에서 보유금액 불러오기
- 2) reset_drink_menu(), reset_snack_menu() 함수를 통해 메뉴 초기화
- 3) check_ID() 함수에서 while문으로 반복
- 4) char mode에 사용자로부터 ID 입력
- 5) if문 통해 "admin", "user"와 비교
- 6) 둘 다 아닐 경우 오류 메시지 출력 후 다시 아이디 입력
- 7) "admin" 혹은 "user"와 일치할 경우 각각 0,1 반환
- 8) main()에서 진입한 모드 출력
- 9) "user"입력을 통해 서비스 모드 접속 시 주문 진행
- 10) 메뉴 목록 화면에 출력 후 음료(1), 간식(2), 구매(3) 선택 옵션 출력 및 입력
- 11) 1 혹은 2 선택 시 메뉴의 ID(id) 통해 영수증에 정보 순서대로 저장
- 12) 메뉴에 없는 번호 선택 시 오류 메시지 출력

- 13) 3번 선택할 때까지 반복
- 14) 재고가 없을 경우 별도 메시지 출력.
- 15) 3 선택 시 지금까지 선택한 메뉴 해당 메뉴의 금액, 그리고 총 금액을 계산하여 화면에 출력.
- 16) 같은 내용의 영수증 receipt.txt에 기록, 저장, 총 금액만큼 보유금액 증가
- 17) (6)에서 "admin"입력 통해 관리자 모드 접속 시 품절된 상품 목록 출력
- 18) 품절된 상품의 종류가 3개 이상이면 재고 주문 수행
- 19) 외부 업체 3곳의 상품 목록(ID, 이름, 가격, 재고) 출력
- 20) 가장 저렴한 상품을 찾아 5개씩 주문. (금액이 모자라다면 목록 상단에 있는 것부터 주문)
- 21) 서비스모드, 관리자모드를 통해 변경된 정보 저장(메뉴 재고, 보유금액)
- 22) 실행 결과

```
Mode Select (admin or user) : admin
Admin mode
===== Sold out Menu =====
1 : Americano 3000 0
8 : Pretzel 3000 0
```

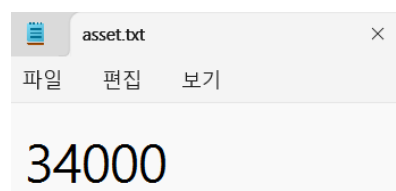
<< 품절된 상품 3개 미만

```
Mode Select (admin or user) : user
User mode
===== drink Menu =====
1 : Americano 3000 0
2 : Cafe Latte 4000 12
3 : Cappuccino 4500 10
4 : Green Tea 2500 8
5 : Black Tea 2500 5
6 : Lemonade 4000 7
7 : Orange Juice 3500 9
8 : Cola 2000 20
9 : Sprite 2000 18

===== snack Menu =====
1 : Cheesecake 5500 5
2 : Chocolate Cake 5500 11
3 : Croffle 4000 1
4 : Muffin 3000 15
5 : Cookie 2500 20
6 : Sandwich 4500 12
7 : Bagel 3500 12
8 : Pretzel 3000 0
9 : Brownie 4000 5

Type 1 for drink, 2 for snack, 3 for buy : 2
Type snack id to buy : 3

Type 1 for drink, 2 for snack, 3 for buy : 3
===== Receipt =====
Croffle 4000
Total : 4000
=====
```



>> 구매 후 보유 금액 늘어남, Croffle 품절로, 품절 상품 3개 됨.

```

Mode Select (admin or user) : admin
Admin mode
===== Sold out Menu =====
1 : Americano 3000 0
3 : Croffle 4000 0
8 : Pretzel 3000 0

===== Gmarket List =====
1 : Americano 2600 80
2 : Cafe Latte 3800 80
3 : Green Tea 2200 80
4 : Black Tea 2200 80
5 : Orange Juice 3200 80
6 : Cola 1900 80
7 : Cheesecake 5200 40
8 : Chocolate Cake 5200 40
9 : Bagel 3200 40
10 : Brownie 3500 40
===== Coupang List =====
1 : Americano 2500 50
2 : Cafe Latte 3500 50
3 : Cappuccino 4000 50
4 : Green Tea 2000 50
5 : Black Tea 2000 50
6 : Lemonade 3500 50
7 : Cheesecake 5000 30
8 : Chocolate Cake 5000 30
9 : Croffle 3500 30
10 : Muffin 2500 30
===== 11st List =====
1 : Americano 2700 100
2 : Cafe Latte 3600 100
3 : Cappuccino 4200 100
4 : Orange Juice 3000 100
5 : Cola 1800 100
6 : Sprite 1800 100
7 : Cookie 2000 50
8 : Sandwich 4000 50
9 : Bagel 3000 50
10 : Pretzel 2500 50

===== Order List =====
Americano 2500 5
Croffle 3500 5
asset : 4000

```

```

Mode Select (admin or user) : user
User mode
===== drink Menu =====
1 : Americano 3000 5
2 : Cafe Latte 4000 12
3 : Cappuccino 4500 10
4 : Green Tea 2500 8
5 : Black Tea 2500 5
6 : Lemonade 4000 7
7 : Orange Juice 3500 9
8 : Cola 2000 20
9 : Sprite 2000 18

===== snack Menu =====
1 : Cheesecake 5500 5
2 : Chocolate Cake 5500 11
3 : Croffle 4000 5
4 : Muffin 3000 15
5 : Cookie 2500 20
6 : Sandwich 4500 12
7 : Bagel 3500 12
8 : Pretzel 3000 0
9 : Brownie 4000 5

Type 1 for drink, 2 for snack, 3 for buy : |

```

>> 재고 주문 후 채워짐, 보유금액 부족으로 Pretzel 재고 안 채워짐

4. 제약 사항

- 1) char mode 최대 길이가 10. 더 긴 ID입력에 문제가 있음
- 2) receipt.txt에 저장되는 양식 보다 가독성 있게 변경할 필요 있음
- 3) 외부업체 목록을 불러와 저장할 보다 효율적인 방법 찾을 필요 있음. 계속 해서 목록을 반복문으로 돌릴 경우 무리가 갈 수 있다고 생각됨.

- 4) 각 서비스 종료 후, 다른 서비스 이용 가능하도록 옵션을 만든다면 더욱 편리할 것으로 생각됨